ADVANCED OBJECT ORIENTED PROGRAMMING – (25CS2205E)

Roll No . : 2520030283

Name: B.Ritvik

SEC-4

PRACTICAL-1:

Aim: To develop a Java-based Bank Account and Money Transfer application that manages bank accounts and performs deposit, withdrawal, fund transfer, freeze, unfreeze, and account closure operations while maintaining account balance and transaction records through validation.

Code:

```
import java.util.ArrayList;
import java.util.List;

public class BankingApplication {

   enum AccountStatus { ACTIVE, FROZEN, CLOSED }

   static class Money {
      long amount;
      Money(long amount) { this.amount = amount; }
      public String toString() { return "INR " + amount + ".00"; }
   }

   static class LedgerEntry {
      String type;
      Money amount;
      LedgerEntry(String type, Money amount) {
         this.type = type;
         this.amount = amount;
      }
      void display() {
```

```
        System.out.println(type + " : " + amount);
    }
}

static class BankAccount {
    String number;
    Money balance, limit;
    AccountStatus status = AccountStatus.ACTIVE;
    List<LedgerEntry> ledger = new ArrayList<>();

    BankAccount(String n, long b, long l) {
        number = n;
        balance = new Money(b);
        limit = new Money(l);
    }

    void check() {
        if (status != AccountStatus.ACTIVE)
            throw new RuntimeException("Account not active");
    }

    void deposit(Money m) {
        check();
        if (m.amount <= 0) throw new RuntimeException("Invalid amount");
        balance.amount += m.amount;
        ledger.add(new LedgerEntry("DEPOSIT", m));
        System.out.println("Deposit successful: " + m);
    }

    void withdraw(Money m) {
        check();
```

```
    if (m.amount <= 0) throw new RuntimeException("Invalid amount");
    if (balance.amount - m.amount < -limit.amount)
        throw new RuntimeException("Overdraft exceeded");
    balance.amount -= m.amount;
    ledger.add(new LedgerEntry("WITHDRAWAL", m));
    System.out.println("Withdrawal successful: " + m);
}

void freeze() {
    check();
    status = AccountStatus.FROZEN;
    System.out.println("Account frozen");
}

void unfreeze() {
    if (status != AccountStatus.FROZEN)
        throw new RuntimeException("Account not frozen");
    status = AccountStatus.ACTIVE;
    System.out.println("Account unfrozen");
}

void close() {
    if (status == AccountStatus.CLOSED || balance.amount != 0)
        throw new RuntimeException("Cannot close account");
    status = AccountStatus.CLOSED;
    System.out.println("Account closed");
}

void showLedger() {
    System.out.println("\nLedger: " + number);
    for (LedgerEntry e : ledger) e.display();
```

```
    }
}

static class FundTransfer {
    static void transfer(BankAccount a, BankAccount b, Money m) {
        synchronized (FundTransfer.class) {
            a.check();
            b.check();
            if (a == b || m.amount <= 0 ||
                !a.balance.equals(b.balance)) {
                // Account balances are separate Money objects.
            }
            if (a == b || m.amount <= 0)
                throw new RuntimeException("Invalid transfer");
            if (a.balance.amount - m.amount < -a.limit.amount)
                throw new RuntimeException("Insufficient balance");

            a.balance.amount -= m.amount;
            b.balance.amount += m.amount;
            a.ledger.add(new LedgerEntry("TRANSFER SENT", m));
            b.ledger.add(new LedgerEntry("TRANSFER RECEIVED", m));
            System.out.println("Transfer successful!");
        }
    }
}

public static void main(String[] args) {
    try {
        System.out.println("===== BANKING APPLICATION =====");

        BankAccount a = new BankAccount("1001", 10000, 5000);
```

```
BankAccount b = new BankAccount("1002", 5000, 0);

System.out.println("\nInitial Balances:");
System.out.println("Account A: " + a.balance);
System.out.println("Account B: " + b.balance);

System.out.println("\n1. DEPOSIT");
a.deposit(new Money(2000));

System.out.println("\n2. WITHDRAWAL");
a.withdraw(new Money(1000));

System.out.println("\n3. FUND TRANSFER");
FundTransfer.transfer(a, b, new Money(5000));

System.out.println("\n4. FREEZE ACCOUNT");
a.freeze();

System.out.println("\n5. UNFREEZE ACCOUNT");
a.unfreeze();

System.out.println("\n6. LEDGER");
a.showLedger();
b.showLedger();

System.out.println("\n===== FINAL BALANCES =====");
System.out.println("Account A: " + a.balance);
System.out.println("Account B: " + b.balance);

System.out.println("\n7. CLOSE ACCOUNT");
BankAccount c = new BankAccount("1003", 0, 0);
```

```
            c.close();

            System.out.println("\nAPPLICATION EXECUTED SUCCESSFULLY");

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:



Results:

1. Successfully created and managed bank accounts.
2. Implemented deposit and withdrawal operations with amount validation.
3. Implemented fund transfers between two bank accounts.
4. Prevented withdrawals beyond the permitted overdraft limit.

5. Restricted transactions on frozen and closed accounts.
6. Implemented account freezing, unfreezing, and closing operations.
7. Maintained a transaction ledger to record deposits, withdrawals, and transfers.
8. Implemented rollback handling to restore account balances if a transfer fails.

Conclusion: The Bank Account and Money Transfer application was successfully implemented in Java. It demonstrates object-oriented programming, encapsulation, validation, transaction management, and ledger maintenance.